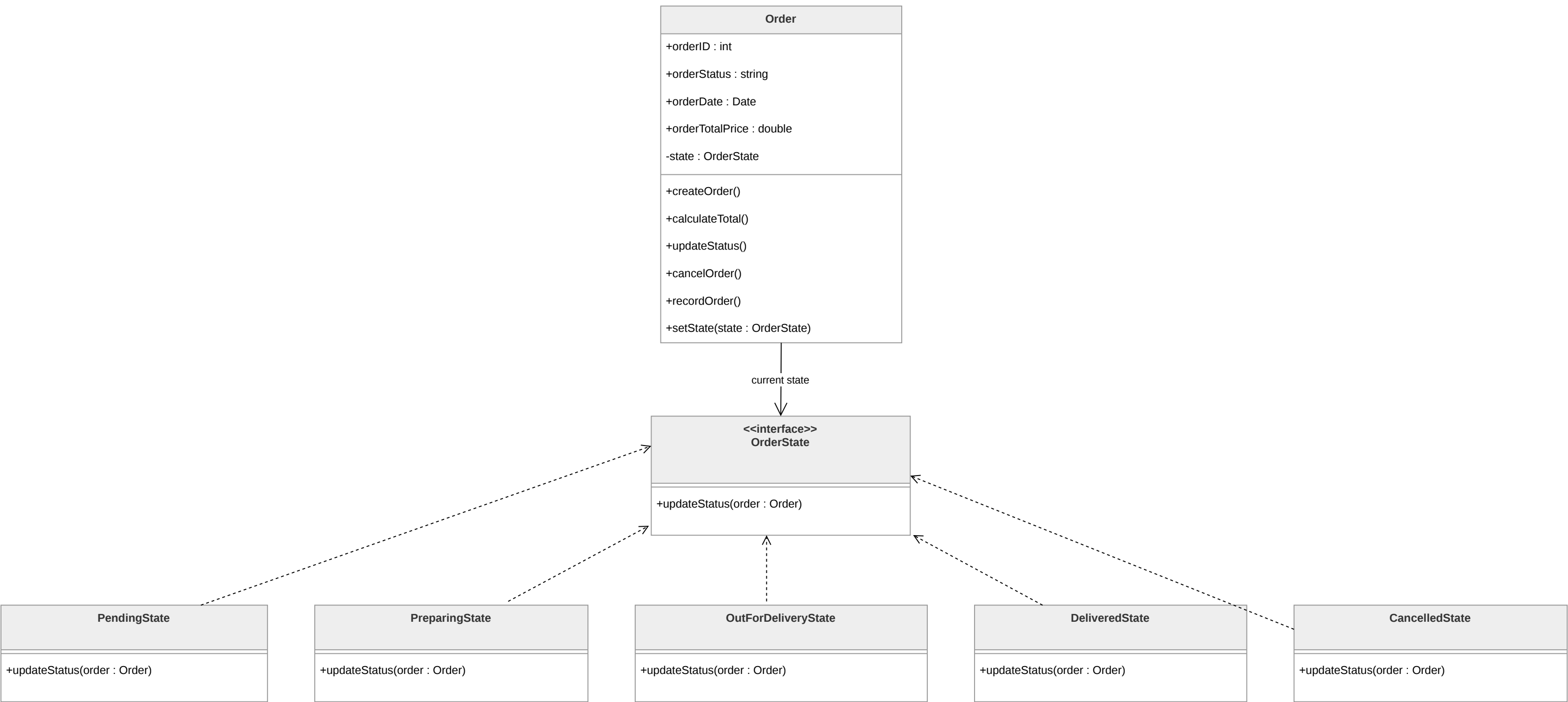


State Design Pattern – Order Status



The State Design Pattern is implemented in the `Order` class to manage the different stages of an order in the food delivery system. Since an order changes its behavior depending on its current status (such as Pending, Preparing, Out for Delivery, Delivered, or Cancelled), the State pattern provides a structured and flexible way to handle these transitions. In this design, the `Order` class acts as the context class, containing a reference to the `OrderState` interface through the `state` attribute. Instead of using multiple `if-else` statements to check the order status, each state is represented by a separate class (`PendingState`, `PreparingState`, `OutForDeliveryState`, `DeliveredState`, and `CancelledState`). Each state class implements the `updateStatus()` behavior differently based on the current order phase. The `OrderState` interface defines the common behavior for all order states, while concrete state classes manage how the order behaves and transitions between statuses. This improves the system by making it more organized, scalable, and easier to maintain, especially when new order statuses need to be added in the future. Overall, the State pattern increases flexibility, readability, and maintainability by separating status-specific logic into independent classes instead of keeping all logic inside the `Order` class.